Terminal-Scroll bricht nach Layout-Switch

Multi-Agent Ultrathink-Analyse

4 Subagenten

Git-Archäologie

Root-Cause bestätigt

30 Slides

Projekt: AgenticExplorer v1.6.26 · **Branch:** master · **Datum:** 2026-04-23

User-Originalzitat:

> „Das Scrollen wenn wir die App starten zu Beginn geht es, wenn ich auf Grid gehe und wieder hin und zurück dann breakt das irgendwann. Ich habe es glaube ich schon 10 mal versucht, vielleicht sollten wir doch ein Refactoring machen."

Phase 1 Output: Root-Cause identifiziert, drei Fix-Optionen mit Bewertung. **STOP — User-Review, keine Implementation ohne Freigabe.**

Executive Summary

- **Root-Cause ist architektonisch, nicht ein Timing-Bug:** `SessionManagerView.tsx:80` enthält die Ternary `layoutMode === "single" ? <single-tree> : <grid-tree>` — zwei **disjunkte JSX-Bäume**.
- **Bei jedem Layout-Switch** unmounted React den einen Baum komplett, mountet den anderen. `term.dispose()` feuert für **jedes Terminal**. Scrollback + Event-Listener + Tauri-Unlisten-Promises verlieren sich.
- **Commit `8b820f5` (Always-Mounted)** hat **den Tab-Switch-Bug** gelöst — NICHT den Layout-Switch. Das Pattern wirkt nur innerhalb des Single-Trees.
- **Warum „irgendwann" und nicht sofort:** async `unlistenPromise.then(...)` in Cleanup (SessionTerminal.tsx:186) erzeugt Zombie-Listener, die sich über mehrere Switches akkumulieren. Plus: Frischer Scrollback fühlt sich im ersten Augenblick "normal" an.
- **Empfehlung:** Option B (CSS-Grid-Layout in einem Baum) — strukturell sauber, eliminiert den Remount, ca. 150 LOC Änderung. Keine globalen Register, keine Portale.
- **Risiko Symptom-Fix-Pfad:** Wenn wir weiter punktuell patchen, landen wir bei Iteration 11 des gleichen Bugs.

Agenda

#	Kapitel	Slides
1	Symptom + Reproduktion	4–5
2	Git-Archäologie: 10 Versuche	6–8
3	Architektur-Mechanik (die JSX-Ternary)	9–13
4	Was beim Layout-Switch konkret passiert	14–17
5	Sekundäre Effekte (PTY-Desync, Zombies)	18–20
6	Warum „irgendwann"	21
7	Agent-Konvergenz + Korrektur	22–23
8	Drei Fix-Optionen	24–27
9	Empfehlung + nächste Schritte	28–30

1 · User-Symptom + Bedingung

- Start: Scrollen funktioniert (frische Terminals, frischer Scrollback-Puffer).
- Switch auf **Grid**-Layout → alte Terminals werden vernichtet, neue aufgebaut.
- Switch zurück auf **Single** → Grid-Terminals werden vernichtet, Single-Terminals werden **neu aufgebaut** (nicht das gleiche Instanz wie am Anfang).
- **Nach N solchen Zyklen bricht Scrollen** — meist 3–10 Wechsel. Unscharf, weil abhängig von aktivem Output, Timing, Tauri-Event-Queue-Tiefe.
- Keine Fehlermeldung in der Konsole sichtbar (Zombie-Listener schlucken via `.catch(() => {})`).

2 · Reproduktions-Schritte

1. `npm run tauri dev` starten.
2. 2–3 Sessions aufmachen (z.B. `pwsh` in unterschiedlichen Ordnern), bis ordentlich Output im Scrollback ist.
3. Im Toolbar `Single` → `Grid` klicken.
4. `Grid` → `Single` klicken.
5. Schritt 3 + 4 wiederholen, **mit dazwischen etwas scrollen** (Mausrad hoch).
6. **Beobachtung:** Nach 3–8 Wechseln bricht das Scroll-Verhalten — entweder
   - Scroll-Rad macht nichts mehr (Canvas inaktiv),
   - Scrollback-Historie ist abrupt „leer" obwohl Bytes noch fließen,
   - `term.write()` triggert aber Viewport bleibt stehen.

Reproduzierbarkeit: **deterministisch**, aber Anzahl Switches bis Break variiert.

3 · Git-Archäologie: Die 10 Versuche

#	Commit	Datum	Fix-Versuch	Gehalten?
1	`9e769b0`	2026-04-06	`overflow:hidden`, debounced ResizeObserver, userScrolledUpRef	teilweise
2	`516eff3`	2026-04-14	Scroll-Track 150ms Delay, Ctrl+C/V Custom-Handler	teilweise
3	`06656e9`	2026-04-15	Revert Ctrl+V (Doppel-Paste)	Regression-Fix
4	`b0e5fe3`	2026-04-16	Error-Logging (.catch → logError)	kein Scroll-Effekt
5	`c098747`	2026-04-22	`document.fonts.ready` vor Initial-Fit	teilweise
6	`8b820f5`	2026-04-23	**Always-Mounted Terminals** (Tab-Switch-Fix)	nur Tab-Switch
7	`c8d64d3`	2026-04-23	Regression-Test zu #6	absichert #6

Plus diverse kleine Refactorings in SessionStore & useSessionEvents.

Muster: Jeder Fix adressierte ein Symptom (Timing, Fit-Race, Cleanup-Reihenfolge). **Kein Fix adressierte bisher den Layout-Switch-Remount selbst.**

4 · Der Eureka-Moment von `8b820f5` — und warum er unvollständig war

Commit-Message: „Scroll-Verlauf überlebt Tab-Switch (always-mounted Terminals)".

Was der Fix wirklich tut (SessionManagerView.tsx:94-104):

```
{activeSessionId ? (
  sessions.map((session) => (
    <div
      key={session.id}
      data-session-wrapper={session.id}
      className="absolute inset-0"
      style={{ display: session.id === activeSessionId ? "block" : "none" }}
    >
      <SessionTerminal sessionId={session.id} />
    </div>
  ))
) : …
```

- **Gut:** Im Single-Mode bleiben alle Terminals gemountet, CSS toggled Sichtbarkeit → Tab-Switch (A→B→A) bewahrt Scrollback.
- **Nicht adressiert:** Dieser ganze Block lebt **innerhalb** des `layoutMode === "single"`-Zweigs. Der andere Zweig (`<SessionGrid>`) ist ein völlig separater Teilbaum.

5 · Architektur-Mechanik: Die Ternary in Zeile 80

```
// SessionManagerView.tsx:80-148 (gekürzt)
{layoutMode === "single" ? (
  previewFolder ? <FavoritePreview …/> : (
    <div ref={containerRef}>
      {/* SINGLE-TREE: sessions.map → <SessionTerminal> always mounted */}
      {sessions.map(s => <div key={s.id}><SessionTerminal sessionId={s.id}/></div>)}
    </div>
  )
) : (
  <div ref={containerRef}>
    {/* GRID-TREE: <SessionGrid> → <GridCell> → <SessionTerminal> */}
    <SessionGrid sessionIds={gridSessionIds} … />
  </div>
)}
```

Das Problem: `<SessionTerminal>` steht in **zwei JSX-Positionen** — in zwei Teilbäumen, die sich gegenseitig **ersetzen**. React hat keine Möglichkeit, eine Instanz von Baum A nach Baum B zu übertragen. Es ist strukturell wie zwei unterschiedliche Bäume auf Top-Level.

6 · Was React bei einem Ternary-Switch macht

```
Vorher (single):                        Nachher (grid):
<div> ── <SessionTerminal sess-A>       <div> ── <SessionGrid>
      ├─ <SessionTerminal sess-B>              ├─ <GridCell sess-A>
      └─ <SessionTerminal sess-C>              │    └─ <SessionTerminal sess-A> ← NEU
                                               ├─ <GridCell sess-B>
                                               │    └─ <SessionTerminal sess-B> ← NEU
                                               └─ <GridCell sess-C>
                                                    └─ <SessionTerminal sess-C> ← NEU
```

React-Reconciler: Top-Level-Node ist in beiden Fällen `<div>`, aber Kinder sind strukturell unterschiedlich. Reconciler unmountet **ALLE** Single-Kinder (inkl. `<SessionTerminal>`-Komponenten) und mountet dann **ALLE** Grid-Kinder neu.
→ `useEffect`-Cleanup in SessionTerminal (Zeile 182-191) feuert für jedes Terminal.

7 · Die fatale Cleanup-Sequenz

```
// SessionTerminal.tsx:182-191
return () => {
  clearTimeout(initialTimer);
  clearTimeout(scrollTrackTimer);
  debouncedFit.cancel();
  unlistenPromise.then((unlisten) => unlisten()).catch(() => {}); // ⚠️ ASYNC!
  scrollDisposable.dispose();
  resizeObserver.disconnect();
  term.dispose();   // ← Zerstört xterm-Instanz + Scrollback-Buffer
  terminalRef.current = null;
};
```

- `term.dispose()` ist **synchron** — xterm-Instanz weg, Canvas weg, Buffer weg.
- `unlistenPromise.then(unlisten → unlisten())` ist **async** — der Tauri-Listener wird erst irgendwann in Zukunft entfernt.
- Der `.catch(() => {})` schluckt Fehler lautlos.

8 · Scrollback ist weg — aber es geht noch weiter

- Der neue Baum mountet mit frischen `<SessionTerminal>`-Instanzen.
- Jede ruft `new Terminal({scrollback: 5000})`, `.open()`, `fitAddon.fit()`, `listen("session-output")`.
- Die **alte** `unlistenPromise` ist vielleicht noch unterwegs. Bis sie fertig ist, existieren **zwei Listener** für dieselbe Session: der alte (auf disposed term) + der neue.
- Alter Listener ruft `term.write()` auf **disposed** Instanz → wirft intern, wird via `.catch(() => {})` verschluckt. Aber: CPU-Zeit verbraucht, Event-Loop belegt.

9 · Beweis: Zwei Mount-Orte

Mount-Ort	Datei:Zeile	Kontext
Single-Tree	`SessionManagerView.tsx:102`	`<SessionTerminal sessionId={session.id} />` in `sessions.map`
Grid-Tree	`GridCell.tsx:99`	`<SessionTerminal sessionId={sessionId} />` in jeder Grid-Zelle

Bei Layout-Switch:

- Single → Grid: **N SessionTerminals im Single-Tree disposed** (wobei N = Gesamtzahl Sessions), **M SessionTerminals im Grid-Tree gemountet** (wobei M ≤ 4, Grid-Limit).
- Grid → Single: **M SessionTerminals disposed**, **N neu gemountet**.

Jeder Switch = komplette dispose/remount-Welle.

10 · xterm-Instanz-Lifecycle im Detail

```
// SessionTerminal.tsx:59-79
useEffect(() => {
  if (!containerRef.current) return;
  const term = new Terminal({ scrollback: 5000, … });   // ← Neue Instanz jedesmal
  const fitAddon = new FitAddon();
  term.loadAddon(fitAddon);
  term.loadAddon(new WebLinksAddon());
  term.open(containerRef.current);                       // ← Canvas binden
  if (containerRef.current.offsetWidth > 0 …) fitAddon.fit();
  terminalRef.current = term;
  …
}, [sessionId, isAtBottom]);
```

- `new Terminal()` **pro Mount**. Scrollback-Puffer ist an diese Instanz gebunden.
- `.dispose()` gibt die Instanz frei. Keine Restore-Logik.
- **Kein globales Register** — es gibt keine Map `sessionId → Terminal`, die Layout-Switches überdauern würde.

11 · Beweis: resize_session-Desync (D1)

Nach jedem neuen Mount:

1. `fitAddon.fit()` berechnet Cols/Rows aus DOM-Dimensionen.
2. `invoke("resize_session", { id, cols, rows })` informiert Rust-PTY.
3. `invoke` ist **async, nicht awaited**.
4. Zwischenzeitlich können neue `session-output`-Events auf alten PTY-Dimensionen kommen.

Quelle: `SessionTerminal.tsx:152-162` · `src-tauri/src/session/manager.rs:372-391`

Konsequenz: Selbst wenn Scrollback gerettet wäre, könnten Zeilen falsch wrappen, weil PTY kurzzeitig andere Cols hat als xterm annimmt.

12 · Warum „irgendwann" und nicht „sofort"

Mehrere additive Effekte pro Switch:

1. **Scrollback-Verlust** — sofort (bei jedem Switch). Der User bemerkt's oft nicht, weil neue Ausgabe kommt.
2. **Zombie-Tauri-Listener** — akkumulativ. Nach N Switches: N-1 Zombies pro Session, die schwach CPU verbrauchen.
3. **PTY-Resize-Rennen** — verstärkt sich nach jedem async invoke, der noch in Flight ist, während schon der nächste abgesetzt wird.
4. **Event-Loop-Druck** — irgendwann kommt ResizeObserver zu spät, `fitAddon.fit()` skippt wegen `offsetWidth === 0`, Scroll-Position landet auf stalem baseY.
5. **Memory** — mehrere xterm-Instanzen warten auf GC.

Schwelle ist nicht konstant (Timing-abhängig) → User empfindet es als „irgendwann".

13 · Agent-Konvergenz: Was bestätigt wurde

Agent	Hauptthese	Status nach Verifikation
D1 (Rust-Backend)	PTY-Resize-Desync via async invoke	**Sekundär bestätigt** — echt, aber nicht Kernursache
D2 (Frontend-Layout)	Zwei Mount-Orte für SessionTerminal	**Primär bestätigt** (seriell, nicht parallel)
D3 (xterm-Lifecycle)	`term.open()` nicht idempotent bei Display-Toggle	**Falsche Fährte** — die Komponente remountet wirklich, `.open()` wird neu gerufen
D4 (Git-History)	10 Symptom-Fixes, `8b820f5` war Teil-Eureka	**Bestätigt** — aber Teil-Eureka für Tab-Switch, nicht Layout

14 · Anti-Hypothesen (was NICHT das Problem ist)

Behauptung	Warum nicht
xterm Canvas „detached" durch CSS `display:none`	Terminal wird bei Layout-Switch wirklich unmounted, also greift Display-Toggle-Theorie nicht
`AnimatePresence` unmountet Terminals	In SessionManagerView/GridCell wird keine AnimatePresence um SessionTerminal gelegt
React-Keys instabil	Keys sind durchgehend `session.id` stabil
Scrollback-Buffer läuft über	5000 Lines = ca. 500 KB, nicht das Limit
Event-Listener akkumulieren **innerhalb** eines Mounts	Pro Mount ein `listen`, bei Unmount wird `unlisten` async gerufen

Der eigentliche Bug ist **strukturell** — kein Timing, kein Config-Value, kein Addon-Problem.

15 · Zusammenfassung der Findings

Root-Cause (bestätigt):

> **Die Ternary `layoutMode === "single" ? <single-tree> : <grid-tree>` in `SessionManagerView.tsx:80` erzwingt einen Komplett-Remount aller `<SessionTerminal>`-Instanzen bei jedem Layout-Switch. xterm-Scrollback ist an die Instanz gebunden und geht dadurch verloren. Zusätzlich akkumulieren async nicht-awaitete Cleanups (Tauri-Unlisten) Zombie-Listener, die das System mit jedem Switch weiter belasten.**

Das ist **nicht** durch punktuelle Patches lösbar. Die drei folgenden Optionen sind strukturell.

16 · Fix-Option A · Globaler Terminal-Registry (extern)

Idee: `Map<sessionId, Terminal>` außerhalb von React. `<SessionTerminal>` holt sich bestehende Instanz aus der Map oder erzeugt eine neue.

```
// hooks/useTerminalInstance.ts (neu, ~40 LOC)
const registry = new Map<string, Terminal>();
export function useTerminalInstance(sessionId: string) {
  return useMemo(() => {
    let t = registry.get(sessionId);
    if (!t) {
      t = new Terminal({ scrollback: 5000, … });
      registry.set(sessionId, t);
    }
    return t;
  }, [sessionId]);
}
```

- **Pro:** Minimale Code-Änderung. Scrollback lebt extern.
- **Contra:** Globalen Zustand einführen. Memory-Leak-Risiko (wer dispose'd die Map?). Schwierig in Tests zu mocken.
- **Aufwand:** ~100 LOC + neue Tests.

17 · Fix-Option B · Ein Baum mit CSS-Grid-Layout (empfohlen)

Idee: Ternary auflösen. Alle `<SessionTerminal>`-Instanzen leben in **einem** stabilen Baum, Layout-Mode steuert nur CSS Grid-Template.

```
// SessionManagerView.tsx (neu skizziert)
<div
  className="h-full w-full grid"
  style={{
    gridTemplateAreas: layoutMode === "single" ? "\"a\"" : gridTemplate,
    gridTemplateColumns: layoutMode === "single" ? "1fr" : "1fr 1fr",
  }}
>
  {sessions.map((s, i) => (
    <div
      key={s.id}
      style={{
        gridArea: layoutMode === "single" ? "a" : GRID_AREAS[i],
        display: layoutMode === "single" && s.id !== activeSessionId ? "none" : "block",
      }}
    >
      <SessionTerminal sessionId={s.id} />
    </div>
  ))}
</div>
```

- **Pro:** Kein Remount, kein globaler State, React-Pattern-konform. Scrollback überlebt auch Layout-Switches. **Löst die Klasse von Bugs, nicht nur den einen.**
- **Contra:** Mehr refaktoring in SessionManagerView. GridCell-spezifische UI-Features (Status-Dot, Maximize-Button) müssen wandern oder konditional gerendert werden.
- **Aufwand:** ~150 LOC + Test-Update (der bestehende Regression-Test c8d64d3 muss erweitert werden auf Layout-Switch).

18 · Fix-Option C · React Portal

Idee: `<SessionTerminal>` rendert in ein Portal, das auf ein stabiles Off-Screen-Div zielt. Layout-Container rendern nur „Mount-Slot"-Divs. Ein kleines Orchestrator-Hook verschiebt den Portal-Target zur richtigen Slot.

- **Pro:** Maximale Flexibilität. Terminals sind komplett unabhängig von Layout-Tree.
- **Contra:** Höchste Komplexität. DOM-Ordnung/Focus/AccessibleName sind Portal-Quellen von Subtil-Bugs. Sehr schwer zu testen.
- **Aufwand:** ~250 LOC + erheblich mehr Test-Schreib-Aufwand.

Option C ist elegant in Präsentationen, aber in Tauri/xterm-Kontexten bisher regelmäßig Quelle neuer Layout-Bugs (eigene Erfahrungswerte). Nicht empfohlen.

19 · Optionen-Vergleich

Kriterium	A (Registry)	B (CSS-Grid)	C (Portal)
Scrollback rettet Layout-Switch	✓	✓	✓
Löst auch Zombie-Listener-Problem	✗	✓	✓
Kein globaler State	✗	✓	✓
React-idiomatisch	○	✓	✗
Test-Aufwand	mittel	mittel	hoch
LOC	~100	~150	~250
Risiko Folgebug	mittel	niedrig	hoch
Verständlichkeit für neuen Dev	mittel	hoch	niedrig

Empfehlung: Option B.

20 · Zusätzlicher Defensive-Fix in Option B

Die Zombie-Listener-Akkumulation kann unabhängig vom Layout-Fix verbessert werden — **und sollte gleich mit**:

```
// SessionTerminal.tsx Cleanup (neu)
return () => {
  clearTimeout(initialTimer);
  clearTimeout(scrollTrackTimer);
  debouncedFit.cancel();
  unlistenPromise.then((unlisten) => {
    try { unlisten(); } catch (e) { logError("SessionTerminal.cleanup.unlisten", e); }
  });   // ⬅️ .catch(() => {}) ersetzen, stumm wird wieder laut
  scrollDisposable.dispose();
  resizeObserver.disconnect();
  term.dispose();
  terminalRef.current = null;
};
```

Auch bei Option B bleibt bestehender async-Cleanup sinnvoll. Kein Ersatz, aber „silent-fail → logged fail" ist wertvoll bei zukünftigen Analysen.

21 · Test-Strategie für Option B

Neue Regression-Tests:

1. **Layout-Switch hält Scrollback:** Vitest mit mocked xterm — `setLayoutMode("grid")` → `setLayoutMode("single")` → `terminalRef.current` ist **dieselbe Instanz** vorher/nachher.
2. **xterm-Instanzen werden nicht neu erzeugt:** Spy auf `new Terminal()` — darf bei Layout-Switch nicht nochmal feuern.
3. **useEffect-Cleanup feuert nicht beim Layout-Switch:** Erweitert `c8d64d3` um Layout-Switch-Pfad.
4. **Tauri-Listener-Count:** Nach 5 Layout-Switches sollten maximal N Listener gleichzeitig registriert sein (N = Sessions).

Integration: `npm run tauri dev` manuell — 20× Layout-Switch mit Live-Output, Scroll bleibt stabil.

22 · Risiken & Mitigationen

Risiko	Wahrscheinlichkeit	Mitigation
`gridSessionIds`-Logik wandert in Render-Layer → Bug-Transfer	mittel	Unit-Test für `setLayoutMode`, der gridSessionIds-Berechnung extrahiert
GridCell-UI (Maximize/Remove) schwer umzubauen	niedrig	Als wrapper um `<SessionTerminal>` erhalten, nur neu positionieren
Neue CSS-Grid-Template führt zu Layout-Flicker	niedrig	Stabiler `grid-template` Wechsel, Framer Motion optional
Bestehender Test `c8d64d3` bricht	hoch	Erwartet — Test wird erweitert/aktualisiert

23 · Feature-Freeze vs. Refactoring

- `CLAUDE.md` sagt: „Feature-Freeze ab v1.6.0 — nur Bugfixes".
- **Dies ist ein Bugfix**, kein Feature. Der User hat den Bug explizit gemeldet, er ist reproduzierbar, er lehnt weitere Symptom-Patches ab.
- Option B ist die **kleinste strukturelle Änderung, die die Klasse von Bugs löst** — darunter geht's nur mit Schmerzen. Das ist ein vertretbares Bugfix-Refactoring.

Alternative „do nothing": Nach Iteration 11 käme Iteration 12, 13, … — das ist für den User erkennbar der schlechtere Weg, er hat explizit gesagt „vielleicht sollten wir doch ein Refactoring machen".

24 · Vorgeschlagener Schritt-für-Schritt-Plan (Option B)

1. **Branch:** `fix/layout-switch-keeps-terminals-mounted`
2. **Commit 1 — Refactor (isoliert):** `refactor(sessions): unify single/grid in one CSS-grid tree` — Ternary entfernen, gridTemplate-Logik einführen, GridCell-spezifische Extras konditional rendern. **Keine Verhaltensänderung** im ersten Commit (Layout-Switch remountet immer noch). Aber SessionTerminals leben jetzt im selben Baum.
3. **Commit 2 — Fix:** `fix(sessions): preserve SessionTerminal instances across layout switches` — die CSS-Grid-Konfiguration nutzen, Keys/Positions konsistent halten. **Hier kippt das Verhalten** — Scrollback überlebt.
4. **Commit 3 — Regression-Test:** `test(sessions): layout-switch preserves xterm instance & scrollback`.
5. **Commit 4 — Defensive-Fix:** `fix(sessions): log unlisten-cleanup errors instead of silent catch`.
6. QA-Gate + `npm run tauri dev` Manual-QA.

25 · Was wir NICHT tun sollten

- **Kein neuer `.fit()`-Timing-Patch.** Die Klasse „wir müssen fit später/früher rufen" ist erschöpft (Commits #1, #2, #5).
- **Keine weiteren ResizeObserver-Wrapper.**
- **Keine Ctrl-Key-Custom-Handler-Änderungen** (Commit #3 hat gezeigt: Bäumeweg, Regressionen anderswo).
- **Kein Versuch, `term.open()` beim Display-Toggle neu aufzurufen** — wie D3 fälschlich vorschlug. Das würde Canvas doppelt binden und neue Bugs erzeugen.
- **Kein „always mounted" für Grid-Mode durch Verschachtelung** — das wäre Option A in ekliger Form.

26 · Metriken für Erfolg

- **Funktional:** Nach 20 aufeinanderfolgenden Layout-Switches bleibt Scrollback lesbar (Scroll-Up zeigt frühere Bytes).
- **Regression-Test:** Vitest-Test „layout-switch preserves xterm instance" grün. Test darf nicht mit Mock-Workaround getrickst werden.
- **Instanzen-Count:** In DevTools `document.querySelectorAll('.xterm').length` bleibt konstant = N (Anzahl Sessions), nicht wachsend.
- **Tauri-Listener-Count:** `window.__TAURI__.event.listeners?.size` (wenn exposed) bleibt stabil.
- **Keine Silent-Errors:** `logError`-Aufrufe mit „cleanup.unlisten" erscheinen nicht wiederholt im Log.

27 · Effekt auf andere Bereiche

- **Kanban-View:** Unbetroffen — hat keinen Terminal.
- **FavoritePreview:** Nutzt eigenes `<ConfigPanel>`, kein Terminal. Unbetroffen.
- **Detached Windows:** Werden via `open_detached_window` in neuem Fenster geöffnet — eigener React-Mount, eigene SessionTerminals, hat mit diesem Bug nichts zu tun.
- **Log Windows:** Ähnlich detached. Unbetroffen.
- **Test-Suite:** `c8d64d3` Regression-Test muss erweitert werden — erwartetes Update, kein Problem.

28 · Offene Fragen an den User (Phase-2-Gate)

1. **Option-Wahl:** A (Registry), **B (CSS-Grid, empfohlen)**, oder C (Portal)?
2. **Scope des Refactors:** Nur SessionManagerView oder auch GridCell mit umbauen?
3. **Defensive-Fix** (silent catch → logError) mit aufnehmen? **Empfohlen: Ja.**
4. **Release-Strategie:** Direkt nach master mergen (v1.6.27), oder erst in Feature-Branch und manuelle QA vor Tag?
5. **Gibt's weitere Symptome**, die wir hier unterschätzt haben? (z.B. Detached-Window + Grid-Switch? Specific Tauri-Version-Abhängigkeit?)

29 · Zeit- & Aufwandsschätzung (Option B)

Schritt	Geschätzt	Agent
Refactor-Commit (Ternary auflösen)	30–45 min	Frontend-Agent
Fix-Commit (CSS-Grid-Template)	20–30 min	Frontend-Agent
Regression-Tests	20–30 min	Test-Agent
Defensive-Cleanup-Fix	5–10 min	Frontend-Agent
QA-Gate (tsc, lint, vitest, build)	10 min	QA-Gate
Manuelle UI-Verifikation	User	Phase 3
Gesamt bis Phase 3	**85–125 min**	—

30 · Entscheidungspunkt — auf dich, Hovsep

Zusammenfassung Phase 1:

- Root-Cause eindeutig lokalisiert: **SessionManagerView.tsx:80 Ternary → doppelter Mount-Tree → dispose bei Layout-Switch**
- Agent-Team hat konvergiert, Widersprüche (D3 Canvas-Theorie) geklärt.
- 10 Symptom-Fixes historisch — weitere Iteration desselben Musters wäre **Zeitverschwendung**.
- Option B ist die kleinste strukturelle Änderung, die die **Bug-Klasse** löst.

Bitte entscheide:

1. **Go Option B?** Dann spawne ich Phase-2-Implementer-Team (F1 Frontend, F2 Tests, F3 QA-Gate). Build + stopp für deine Phase 3.
2. **Lieber Option A (Registry)?** Schneller, aber Band-Aid-Charakter.
3. **Abwarten / anders angehen?** Sag was — ich bleibe stehen.

> Phase 1 ist durch. Phase 2 wartet auf dein Go.